Subject: Logic & Data Migration Strategy – Multi-Domain FinTech Platform
From: Sam (Software Engineer)
To: Architecture & Delivery Teams
Date: 9 Jun 2026

---

1. Executive Summary

Based on the **Architecture Summary Report**, we will migrate the legacy "FinTech Logic" codebase (banking, telecoms, insurance, hospitality, schools) into the new unified platform. The plan preserves domain-specific rules, ensures data integrity, and meets regulatory compliance while using the existing local folder (`C:\Users\Arthur Magaya\Downloads\FinTech Logic`) as the source of truth for business-logic artifacts [1].

2. High-Level Migration Phases

Phase	Duration	Key Deliverables
A – Discovery & Mapping	2 wks	Domain data models, business rules, API contracts, compliance requirements
B – Architecture Alignment	1 wk	Target schema, micro-service boundaries, event-driven patterns
C – Data Migration	4 wks	ETL pipelines (Kafka + Debezium, Airbyte), data validation, cut-over plan
D – Logic Refactor & Service Build	6 wks	Domain services, API gateways, security, monitoring
E – Integration & Testing	3 wks	End-to-end tests, performance, security audits
F – Cut-over & Post-Go-Live	1 wk	Switch-over, rollback plan, knowledge transfer

3. Domain-Specific Logic Overview

Domain	Core Business Logic	Key Data Entities	Compliance Notes
Banking	Transaction validation, AML checks, interest accrual	Accounts, Transactions, Customers	PCI-DSS, KYC
Telecoms	Billing cycles, usage throttling, roaming rules	Subscribers, CallRecords, Plans	GDPR
Insurance	Policy underwriting, claim adjudication, risk scoring	Policies, Claims, Insureds	ISO 27001
Hospitality	Reservation handling, loyalty points, dynamic pricing	Reservations, Guests, Rooms	PCI-DSS
Schools	Enrollment, grading, attendance tracking	Students, Courses, Grades	FERPA

All tables are derived from the Architecture Summary Report and the local "FinTech Logic" folder contents [1].

4. Implementation Details (Software-Engineer Slice)

1. **Codebase Audit** – Clone the `FinTech Logic` folder, run static analysis (ESLint, SonarQube) to catalog domain-specific modules.
2. **Micro-service Extraction** – For each domain, create a separate repository with a clean `Dockerfile` and CI pipeline (GitHub Actions).
3. **Data Pipelines** –
   - Use **Debezium** to capture CDC from legacy databases.
   - Route events through **Kafka** topics named per domain (`banking.transactions` , `telecoms.usage` , etc.).
   - Transform with **dbt** models to align with the target schema.
4. **Schema Migration** – Apply versioned migrations via **Flyway**; store scripts in a `db/migrations` folder per domain.
5. **Testing** – Generate contract tests (Pact) for each API; run integration suites in a staged environment before cut-over.

5. Next Steps

- Verify the exact path and contents of the “FinTech Logic” folder (confirm presence of source files).
- Align on preferred data-pipeline tooling (Kafka vs. Pulsar) and cloud provider for managed services.
- Schedule a walkthrough of the Architecture Summary Report with the Architecture Lead to confirm domain boundaries.

---

*Email sent to **arthurmagaya29@gmail.com** as requested [3].*